

Chapter6 Notes

Unit VI: Client Side Scripting with JavaScript

[8 Hrs.]

1. Introduction to JavaScript

- JavaScript (JS) — programming language that runs in browser, adds **interactivity** to static HTML/CSS pages.
 - Client-side scripting language (also runs server-side via Node.js, but this unit = browser/client context).
 - Real project use: form validation, dynamic content updates, interactive UI (dropdowns, sliders, modals), API calls (fetching data without page reload).
-

2. Using JS in HTML

Inline

```
<button onclick="alert('Clicked!')">Click Me</button>
```

Internal

```
<head>
  <script>
    console.log("Page loaded");
  </script>
</head>
```

External (preferred in real projects)

```
<head>
  <script src="script.js" defer></script>
</head>
```

```
// script.js
console.log("Loaded from external file");
```

Attribute	Purpose
<code>defer</code>	Script runs after HTML fully parsed — used in almost every real project to avoid blocking page render
<code>async</code>	Script runs as soon as downloaded (independent order) — used for analytics/ad scripts

Real project practice: place `<script src="script.js" defer></script>` in `<head>`, or plain `<script>` just before `</body>` — both avoid render-blocking issues common in beginner code.

3. JavaScript Output

Method	Purpose	Real use
<code>console.log()</code>	Print to browser console	Debugging during development
<code>alert()</code>	Popup message box	Simple warnings (rarely used in production UI, mostly debugging)
<code>document.write()</code>	Write directly to page	Avoid in real projects — overwrites entire page if called after load
<code>innerHTML</code>	Insert HTML into element	Dynamically updating page content (most common in real projects)
<code>textContent</code>	Insert plain text into element	Safer than innerHTML when inserting user data (avoids XSS)

```
console.log("Debug: user logged in");

document.getElementById("cartCount").innerHTML = "3 items";
```

```
document.getElementById("username").textContent = userInput; // safer for untrusted input
```

4. JavaScript Comments

```
// single-line comment - explain tricky logic

/* multi-line comment
   used for larger explanations or temporarily disabling code blocks */

// Real project example:
// TODO: replace with API call once backend is ready
let products = ["Shoes", "Watch", "Bag"];
```

5. Variables and Data Types

Declaring variables

```
let username = "Uttam"; // block-scoped, can be reassigned - most used in modern real projects

const taxRate = 0.13; // block-scoped, cannot be reassigned - used for constants
var oldStyle = "avoid"; // function-scoped, legacy - avoided in modern code
```

Real project rule: use `const` by default, `let` only when value will change, avoid `var` entirely (common lint rule in real codebases — ESLint flags `var`).

Data Types

```
let price = 499.99; // Number
let productName = "T-Shirt"; // String
let inStock = true; // Boolean
let discount = null; // Null - intentionally empty
let category; // Undefined - declared but not assigned
let user = { name: "Uttam", age: 25 }; // Object
let cart = ["Shoes", "Watch"]; // Array (special type of object)
```

Real project example — user profile object (common pattern in any app):

```
const user = {
  id: 101,
```

```
name: "Uttam Karna",
email: "uttam@example.com",
isPremium: false
};
console.log(user.name);    // "Uttam Karna"
```

6. Operators and Expressions

Type	Operators	Example
Arithmetic	+ - * / % **	let total = price * qty;
Assignment	= += -= *= /=	total += tax;
Comparison	== === != !== > < >= <=	if (age >= 18)
Logical	&& \ !	if (isLoggedIn && hasPermission)
Ternary	condition ? a : b	let status = age >= 18 ? "Adult" : "Minor";

Important real-project rule: always use `===` / `!==` (strict equality), not `==` / `!=`

```
console.log(0 == "0");    // true - type coercion, source of bugs
console.log(0 === "0");   // false - strict comparison, safer, industry standard
```

Real project example — shopping cart total calculation:

```
const price = 500;
const quantity = 3;
const taxRate = 0.13;

const subtotal = price * quantity;
const tax = subtotal * taxRate;
const total = subtotal + tax;

console.log(`Total: Rs. ${total.toFixed(2)}`);
```

7. Control Flow and Conditionals

```
// Real project: discount logic based on cart total
const cartTotal = 3500;
let discount;
```

```
if (cartTotal >= 5000) {
  discount = 0.20;
} else if (cartTotal >= 2000) {
  discount = 0.10;
} else {
  discount = 0;
}
console.log(`Discount: ${discount * 100}%`);
```

```
// Real project: switch for handling order status display
const orderStatus = "shipped";

switch (orderStatus) {
  case "pending":
    console.log("Your order is being processed.");
    break;
  case "shipped":
    console.log("Your order is on the way!");
    break;
  case "delivered":
    console.log("Order delivered.");
    break;
  default:
    console.log("Unknown status.");
}
```

```
// Ternary - common in React/real projects for conditional rendering logic
const isLoggedIn = true;
const message = isLoggedIn ? "Welcome back!" : "Please log in.";
```

8. Loops

```
// for loop - real project: rendering list of products
const products = ["Shoes", "Watch", "Bag"];
for (let i = 0; i < products.length; i++) {
  console.log(`${i + 1}. ${products[i]}`);
}

// while loop - real project: retry logic for failed API call
let attempts = 0;
while (attempts < 3) {
  console.log("Attempting to connect...");
  attempts++;
}
```

```

// do-while - runs at least once - real project: show popup at least once per session
let shown = false;
do {
  console.log("Showing welcome popup");
  shown = true;
} while (!shown);

// for...of - iterating array values (modern, common in real code)
for (const product of products) {
  console.log(product);
}

// for...in - iterating object keys
const user = { name: "Uttam", age: 25 };
for (const key in user) {
  console.log(`${key}: ${user[key]}`);
}

```

9. Functions

```

// Function declaration
function calculateTotal(price, qty) {
  return price * qty;
}

// Function expression
const applyDiscount = function(total, percent) {
  return total - (total * percent / 100);
};

// Arrow function (modern, widely used in real projects, especially React)
const formatPrice = (amount) => `Rs. ${amount.toFixed(2)}`;

// Default parameters - real project: shipping cost defaults if not provided
function calculateShipping(weight, ratePerKg = 50) {
  return weight * ratePerKg;
}

console.log(calculateTotal(500, 3));           // 1500
console.log(applyDiscount(1500, 10));         // 1350
console.log(formatPrice(1350));               // "Rs. 1350.00"

```

Real project example — reusable validation function used across a signup form:

```

function validateEmail(email) {
  const pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

```

```

    return pattern.test(email);
}

if (!validateEmail(document.getElementById("email").value)) {
    alert("Please enter a valid email address.");
}

```

10. Arrays

```

const cart = ["Shoes", "Watch"];

cart.push("Bag");           // add to end   -> ["Shoes","Watch","Bag"]
cart.pop();                 // remove last  -> ["Shoes","Watch"]
cart.unshift("Cap");        // add to start -> ["Cap","Shoes","Watch"]
cart.shift();               // remove first -> ["Shoes","Watch"]
cart.splice(1, 0, "Belt");   // insert at index 1 -> ["Shoes","Belt","Watch"]
cart.slice(0, 2);            // returns copy of first 2 items, doesn't modify original

console.log(cart.length);    // number of items

```

Iteration methods (heavily used in real projects, especially rendering UI lists)

```

const prices = [500, 1200, 300, 800];

// forEach - just loop, no return value
prices.forEach(p => console.log(p));

// map - transform each item, returns NEW array - real use: rendering product cards
// from data
const withTax = prices.map(p => p * 1.13);

// filter - keep items matching condition - real use: filter products under budget
const affordable = prices.filter(p => p < 1000);

// reduce - combine into single value - real use: calculate cart total
const total = prices.reduce((sum, p) => sum + p, 0);

console.log(withTax);      // [565, 1356, 339, 904]
console.log(affordable);   // [500, 300, 800]
console.log(total);        // 2800

```

Real project example — rendering product list to page dynamically:

```

<ul id="productList"></ul>
<script>

```

```

const products = [
  { name: "Shoes", price: 3000 },
  { name: "Watch", price: 5000 }
];

const list = document.getElementById("productList");
products.forEach(product => {
  const li = document.createElement("li");
  li.textContent = `${product.name} - Rs. ${product.price}`;
  list.appendChild(li);
});
</script>

```

11. Objects

```

const product = {
  name: "Wireless Mouse",
  price: 1500,
  inStock: true,
  tags: ["electronics", "accessories"],
  getDiscountedPrice: function(percent) { // method
    return this.price - (this.price * percent / 100);
  }
};

console.log(product.name); // dot notation
console.log(product["price"]); // bracket notation - needed when key is
                               // dynamic/variable
console.log(product.getDiscountedPrice(10)); // 1350

```

Object constructors / classes (creating multiple similar objects)

```

// Constructor function (older style)
function Product(name, price) {
  this.name = name;
  this.price = price;
}
const p1 = new Product("Shoes", 3000);

// ES6 Class (modern, preferred in real projects)
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
  greet() {

```

```

        return `Hello, ${this.name}`;
    }
}

const user1 = new User("Uttam", "uttam@example.com");
console.log(user1.greet()); // "Hello, Uttam"

```

Prototypes (brief)

- Every JS object has a hidden link to a **prototype** object it inherits methods/properties from.
- This is why arrays have `.map()`, `.filter()` etc. — these are defined on `Array.prototype`.

```

console.log(product.hasOwnProperty("price")); // true - hasOwnProperty comes from
Object.prototype

```

Real project example — combining objects + arrays + DOM (typical small feature: cart summary):

```

<div id="cartSummary"></div>
<script>
const cartItems = [
  { name: "Shoes", price: 3000, qty: 1 },
  { name: "Socks", price: 300, qty: 2 }
];

const total = cartItems.reduce((sum, item) => sum + item.price * item.qty, 0);

document.getElementById("cartSummary").innerHTML = `
  <h3>Cart Summary</h3>
  <ul>
    ${cartItems.map(item => `<li>${item.name} x${item.qty} - Rs. ${item.price *
    item.qty}</li>`).join("")}
  </ul>
  <strong>Total: Rs. ${total}</strong>
`;
</script>

```

This single example mirrors exactly how real e-commerce cart summaries are built with vanilla JS.

Quick Recap (Exam Points)

- JS added to HTML: inline, internal, external (`defer` / `async` attributes).
- `let` / `const` preferred over `var`; `const` = default, `let` = reassignable.
- Data types: Number, String, Boolean, Null, Undefined, Object, Array.
- Use `===` / `!==` (strict) not `==` / `!=` to avoid type coercion bugs.

- Control flow: `if/else if/else`, `switch`, `ternary ? :`.
 - Loops: `for`, `while`, `do-while`, `for...of` (arrays), `for...in` (objects).
 - Functions: declaration, expression, arrow function, default parameters.
 - Array methods: `push/pop/shift/unshift/splice/slice`; iteration: `forEach/map/filter/reduce`.
 - Objects: dot vs bracket notation, methods, constructor functions/classes, prototypes.
-

Possible Exam Questions

Short Answer / Conceptual

1. What is JavaScript? Explain its role in client-side web development.
2. Differentiate between `var`, `let`, and `const`.
3. What is the difference between `==` and `===` in JavaScript? Give an example.
4. List and explain JavaScript's primitive data types with examples.
5. Differentiate between `for` loop and `for...in` loop.
6. What is the difference between function declaration and arrow function?
7. Explain any four array methods (`push`, `pop`, `map`, `filter`) with examples.
8. What is the difference between `map()` and `forEach()`?
9. What is a JavaScript object? How do you access its properties (two ways)?
10. What is a prototype in JavaScript?
11. Differentiate between `innerHTML` and `textContent`.
12. What is the difference between `async` and `defer` attributes in `<script>` tag?

Long Answer / Programming

1. Write a JavaScript function to calculate the total price of items in a shopping cart using the `reduce()` method.
2. Write a program using a `for` loop to display a list of products from an array on a webpage.
3. Create a simple JavaScript form validation script that checks if email and password fields are not empty, and email is in valid format.
4. Write a JavaScript class `Student` with properties `name` and `marks`, and a method to determine pass/fail (`marks >= 40`).
5. Write JavaScript code to filter an array of product prices to show only items below Rs. 1000, and display the result on the page.
6. Demonstrate the use of `map()`, `filter()`, and `reduce()` together on an array of order objects to compute the total revenue from orders above Rs. 500.
7. Write a program that dynamically adds/removes items from a to-do list using array methods (`push`, `splice`) and updates the DOM.

True/False or Fill in the Blanks

1. `const` variables can be reassigned after declaration. (False)
2. `===` checks both value and type, while `==` only checks value. (True)
3. `map()` method modifies the original array. (False — returns a new array)
4. The _____ loop is guaranteed to execute at least once. (`do-while`)
5. _____ keyword is used to create an object from a class in JavaScript. (`new`)
6. Arrow functions were introduced in _____ (ES6/ES2015).